

Projet de synthèse : **Livrable 4**

Enseignants :

- Maria Alice Bertolim

Elèves :

- Matthieu Damien
- Florian Alzaix

Table des matières

1	Heatmaps	3
1.1	Models testés côté deep	3
1.1.a	CNN2 + transformer	3
1.1.b	CNN3D	3
1.2	Implémentations bonus	3
1.3	Problème principal rencontré	4
1.4	Résultats	4
2	Analyse au niveau du pas (step-level)	5
2.1	Construction de l'index de pas (steps.csv)	5
2.2	Architecture de chargement — StepDataset	6
2.3	Comparaison de modèles step-level (model_step_comparison.py)	7
2.3.a	Features tabulaires	7
2.3.b	Modèles évalués	7
2.3.c	Résultats (niveau sujet — majority vote)	7
2.4	Identification des pas anormaux	8
2.5	QC visuel (step_qc.py)	9
2.6	Modifications du pipeline	9
2.7	Scripts ajoutés	9

1 Heatmaps

1.1 Models testés côté deep

Nous avons mis en place deux architectures de deep learning différentes:

- CNN2D + transformer temporel
- CNN3D

Du fait que nous avons en fait un canal temporel en plus du fait que d'avoir des images cela implique le fait d'avoir la capacité de traiter un canal supplémentaire qui est le temps. Par conséquent ce sont souvent deux solutions qui sont utilisées dans ce genre d'applications.

1.1.a CNN2 + transformer

Un CNN permet d'extraire les données d'une image. Dans notre cas il va apprendre à reconnaître les zones de pressions par exemple.

De l'autre côté nous avons le transformer temporel qui va venir faire le lien entre les différentes features qui seront extraites des images par le CNN.

1.1.b CNN3D

Il va extraire directement les motifs locaux mais en considérant la dimension temporelle dans l'extraction. En fait ça vient du fait qu'il fait une convolution en 3d donc en fait il va regarder les motifs locaux et comment il se déplacent dans le temps.

1.2 Implémentations bonus

Nous avons ajouté pour solidifier l'entraînement plusieurs choses:

- Early stopping quand le loss ne diminue plus.
- Un ReduceLROnPlateau pour réduire le lr justement quand le loss ne diminue plus pour pousser un plus la précision.
- Un choix du nombre d'époch en live pour pouvoir ajuster si besoin.

- Un BCEWithLogitsLoss pour rééquilibrer le léger déséquilibre des classes 20% en plus de patients malades dans notre dataset.
- Le tensorboard pour suivre l'évolution du loss et de la précision

Note: nous pourrions pousser les scores pour avoir plus de métrique. Pour le moment nous avons déjà décidé de pousser un plus les techniques pour maximiser la précision afin d'avoir une base solide plutôt que d'affiner les petits détails directement.

1.3 Problème principal rencontré

Nous avons des valeurs qui sortaient constamment les mêmes où le modèle stagnait. Pour changer ça nous avons décidé de changer le prétraitement en enlevant la quantification en uint16 que l'on faisait qui en fait retirait des données. Ce faisant, cela a augmenté la taille sur le disque du nouveau dataset → 7Go.

Cependant après un tel changement nous avons commencé à voir les modèles sortir de ces valeurs répétitives et donc commencer à apprendre une logique.

1.4 Résultats

Nous avons testé les deux modèles avec plein d'hyperparamètres différents.

Le maximum d'accuracy que nous avons obtenue sur un apprentissage stable c'est avec le CNN2D et le transformer → 72%.

Les paramètres principaux étaient:

- lr : 0.0002
- epochs : 20 (mais max atteint à la 6ème)
- weight_decay: 0.0002
- batch_size: 4
- temporal_stride: 1 (donc pas d'échantillonnage des vidéos)
- gait_duration: 5 (5s de vidéos)

- `frame_cnn_chunk_size`: 32
- `dropout`: 0.2

Note: pour améliorer ces résultats il nous faudrait plus de temps pour améliorer les architectures, se pencher sur le côté mathématique derrière les modèles pour avoir des meilleures intuitions.

Note: Aussi une epoch prenait environ 500s au début donc les premiers entraînements étaient très longs. Par la suite nous avons réussi à réduire ce temps en optimisant le chargement des datasets mais encore beaucoup de temps d'entraînement.

2 Analyse au niveau du pas (step-level)

2.1 Construction de l'index de pas (`steps.csv`)

Un index maître `output/steps.csv` a été construit via `project/build_steps_index.py`. Chaque ligne correspond à un pas (phase d'appui) segmenté sur `total_L` ou `total_R`.

Paramètres de segmentation finalisés :

Paramètre	Valeur	Justification
<code>_STANCE_THRESHOLD_RATIO</code>	0.08	Validé empiriquement (réduction 24% des fusions L/R sans sur-segmentation)
<code>_MIN_STEP_SAMPLES</code>	10 (0.1 s)	Garde défensif — les pas < 0.1 s sont des artefacts

<code>_MAX_STEP_SAMPLES</code>	200 (2.0 s)	Artefact de fusion confirmé par QC visuel
<code>_MAX_STRIDE_DISTANCE_SAMPLES</code>	150 (1.5 s)	Validé : maximum observé = 149 samples sur le dataset complet

Tableau 2.1

Statistiques finales (60 439 pas, 165 sujets, 306 enregistrements) :

Flag	Count	%
ok	59 457	98.4 %
edge_start	487	0.8 %
edge_end	383	0.6 %
too_long	112	0.2 %

Tableau 2.2

L'appariement L/R par `stride_id` (algorithme glouton par midpoint) couvre la quasi-totalité des pas. Les 112 pas `too_long` correspondent à des fusions d'appuis ou des pauses, confirmés visuellement via `qc_long_strides.py`.

2.2 Architecture de chargement — StepDataset

`project/step_dataset.py` implémente un dataset lazy-loading compatible PyTorch (`__len__` + `__getitem__`):

- Filtre par défaut: exclut `too_short` et `too_long` → **60 327 pas** actifs
- Retourne le signal brut (`T`, 16) (16 capteurs individuels L1-L8, R1-R8)
- `SubjectSplitter` encapsule `GroupKFold` pour garantir que tous les pas d'un sujet restent dans le même fold

2.3 Comparaison de modèles step-level (model_step_comparison.py)

2.3.a Features tabulaires

7 features par pas, calculées dans `build_step_features()` :

Feature	Description
<code>duration_s</code>	Durée de la phase d'appui
<code>peak_force</code>	Force maximale
<code>auc_norm</code>	$\text{auc_force} / \text{duration_s}$ — force moyenne sans biais de durée
<code>foot_L</code>	Encodage binaire du pied
<code>stride_asym_peak</code>	$ \text{peak_L} - \text{peak_R} / (\text{peak_L} + \text{peak_R})$
<code>stride_asym_duration</code>	Asymétrie de durée intra-foulée
<code>stride_asym_auc</code>	Asymétrie d'AUC intra-foulée

Tableau 2.3

Les 3 features d'asymétrie sont NaN pour les pas non appariés (~5 %), imputées par médiane dans les pipelines.

2.3.b Modèles évalués

RF, LogReg, KMeans (K=2), Fuzzy C-Means (C=2), CNN 1D PyTorch — tous évalués par **GroupK-Fold 5 folds par sujet** pour éviter la fuite de données.

2.3.c Résultats (niveau sujet — majority vote)

Modèle	Balanced Acc	ROC-AUC
FCM	0.69	0.75

KMeans	0.56	0.70
LogReg	0.56	0.78
RF	0.56	0.48
CNN 1D	0.60	0.73

Tableau 2.4

Ces résultats sont produits sur un sous-ensemble de 30 sujets (15 PD + 15 CO) pour les tests de développement. Les valeurs sur le dataset complet seront obtenues à l'exécution de `run_step_comparison()`.

FCM est le modèle le plus robuste au niveau sujet, cohérent avec les résultats du clustering flou du rapport 3 sur les features agrégées. La performance step-level reste modeste (Balanced Acc ~0.55–0.65), ce qui est attendu : un pas individuel est peu discriminant, c'est l'agrégation par sujet qui porte le signal.

2.4 Identification des pas anomaux

`build_focus_table()` produit `output/etude_du_pas/focus_malades_premiers_pas.csv` avec trois types :

Type	Condition	Interprétation
pd_mismatch	Sujet CO, RF prédit PD	Pas PD-like chez un sujet sain — indicateur précoce potentiel
high_asym	<code>stride_asym_peak > 0.15</code> ou <code>stride_asym_duration > 0.10</code>	Asymétrie pathologique (seuil Robinson 1987)
both	Cumul des deux	Cas les plus saillants cliniquement

Tableau 2.5

2.5 QC visuel (step_qc.py)

project/step_qc.py génère une figure par pas sélectionné (top 20 par type, tous les both) :

- **Panneau 1** : vue contextuelle ± 4 s avec le pas cible surligné et le pas partenaire de foulée grisé
- **Panneau 2** : zoom sur le pas avec textbox métriques (y_prob, asymétries, durée, peak)

Le QC visuel confirme que la majorité des cas pd_mismatch et high_asym sont physiologiquement plausibles et non des artefacts de segmentation.

Sorties dans output/figures/step_qc/ + output/etude_du_pas/step_qc_summary.csv .

2.6 Modifications du pipeline

- main.py : passage de 9 à **10 étapes**, project.step_qc.main() ajouté en étape 10
- project/config.py : STEP_QC_FIG_DIR ajouté
- output/etude_du_pas/figures/model_step_comparison/model_comparison_table.csv : tableau de comparaison mean $\pm$ std exporté automatiquement

2.7 Scripts ajoutés

Script	Rôle
project/build_steps_index.py	Construit steps.csv et steps_features.csv
project/step_dataset.py	Dataset lazy-loading + SubjectSplitter
project/model_step_comparison.py	Comparaison RF / KMeans / FCM / CNN 1D step-level
project/step_qc.py	QC visuel des pas pd_mismatch et high_asym
project/qc_long_strides.py	QC visuel des strides avec distance L/R > 1 s

Tableau 2.6